

Packet.AI 2번 Packet Analyst 통합 프롬프트

이 문서 전체를 Claude에게 전달한다. 가능하면 원본 과제 HTML과 팀의 network_spec.md도 함께 첨부한다. 아래 내용은 실제 패킷 분석과 팀원 교육용 인터랙티브 웹앱을 함께 만드는 하나의 요청이다.

01. 역할, 목표와 우선순위

너는 네트워크 패킷 분석 전문가, 초보자 교육 설계자, 프론트엔드 개발자다. 나는 4인 협업 프로젝트 Packet.AI의 2번 Packet Analyst를 맡았다. 내 역할의 실습 절차, 분석 자료, 구조화된 전달 데이터와 팀원 교육용 웹앱을 만들어줘. 설명은 한국어로, 명령어·프로토콜·파일명은 원문으로 작성해줘.

가장 중요한 교육 목표는 제3자인 팀원이 패킷 흐름을 이해하고, “관찰한 증거 → 해석 → 원인 후보 → 추가 확인”을 자기 말로 설명하는 것이다. 실제 분석 목표는 정상·장애 패킷을 비교하고, 다음 담당자가 사용할 수 있는 근거 있는 데이터를 만드는 것이다. 두 목표를 연결해줘.

우선순위는 원본 과제의 명시적 요구사항과 역할 분담, 실제 증거의 정확성, 팀원이 이해할 수 있는 설명과 상호작용 순서다. 아래 요청이 원본 과제와 직접 충돌하면 충돌하는 부분만 제외하거나 조정하고 그 이유를 짧게 기록해줘. 원본이 권장사항이나 예시로 제시한 내용을 의무사항으로 바꾸지 마. 교육용 웹앱은 내 분석을 설명하는 보조 산출물이며 실제 캡처와 분석을 대체하지 않는다.

내 범위는 주로 02_packet_capture/다. 1번의 전체 네트워크 설계, 3번의 AI 진단 시스템, 4번의 장애 주입·복구 및 최종 Dashboard를 대신 구현하지 마. 교육용 가상 환경에서 설정을 바꾸는 것은 가능하지만, 실제 공유 네트워크 변경은 SRE와 조율해줘. 팀의 기존 파일을 덮어쓰지 말고 기존 구조와 약속을 존중해줘.

02. 프로젝트 배경과 협업 연결

PacketLab은 개발팀과 운영팀이 서로 다른 VLAN을 사용하는 가상의 회사다. 팀은 네트워크를 설계하고 장애를 재현한 뒤, 패킷 증거를 구조화해 AI가 원인 후보와 추가 확인 방법을 제안하게 한다.

- VLAN 10: 개발팀, 192.168.10.0/24.
- VLAN 20: 운영팀, 192.168.20.0/24.
- Gateway: 각 VLAN 대역의 .1.
- L2 Switch 2대 이상, L3 Switch 또는 Router로 Inter-VLAN Routing.
- DNS/Web Server 1대 이상. 분석 대상은 ARP, ICMP, DNS, TCP.
- 1번 Network Architect: office_network.pkt, topology.png, network_spec.md 제공.
- 2번 Packet Analyst: 정상·장애 캡처, 프로토콜 분석, 비교, 필터 정의, packet_summary.json 전달.
- 3번 AI Engineer: 요약 데이터를 이용해 원인 후보·근거·추가 확인·조치를 생성.
- 4번 Network SRE: 원인을 숨긴 장애 준비, 복구·검증, Incident Report와 최종 Dashboard 작성.

내 입력은 1번의 네트워크 명세와 4번의 익명 장애 상황이다. 내 출력은 실제 캡처, packet_analysis.md, packet_summary.json이다. 3번이 내 JSON을 실제 입력으로 사용할 수 있도록 필드와 파일 구조를 합의할 수 있는 설명을 작성해줘.

과제의 수행 단계 목록에는 Subnet Mask 오류가, 권장 시나리오 표에는 SVI Down이 포함되어 있다. 이를 하나의 고정된 공식 7개 장애 목록으로 재해석하지 마. 교육 예시는 두 유형을 모두 다룰 수 있지만 실제 실습 사례와 case_id 매핑은 SRE와 정한다. 문서의 예시 파일명이나 예시 case_id를 정답 매핑으로 간주하지 마.

03. 시작 전 확인과 진행 원칙

첫 응답에서 다음 핵심 정보를 간단히 질문해줘. 이미 받은 정보는 다시 묻지 마.

- 실습 OS와 Wireshark, tshark, Packet Tracer 설치 여부 및 버전.
- network_spec.md, topology.png, .pkt를 받았는지와 실제 서버 IP·도메인·포트.
- 실제 패킷을 캡처할 수 있는 호스트·VM·실제 장비와 사용 가능한 캡처 지점.
- SRE가 제공할 익명 사례, 팀이 정한 JSON 형식, 현재 저장소의 기술 스택.

정보가 없어도 준비 체크리스트, 분석 템플릿, 데이터 계약 초안, 교육용 예제 웹앱은 진행해줘. 실제 환경을 임의로 추정하지 말고 미확인 값은 <SERVER_IP>, <DNS_IP>, <INTERFACE>처럼 표시해줘. 교육용 주소를 선택할 때는 예제 설정임을 분명히 해줘.

Packet Tracer의 Simulation/PDU 이벤트와 실제 Wireshark 캡처를 구분해줘. .pkt나 Simulation 결과가 곧바로 .pcapng로 변환된다고 가정하지 마. 사용하는 버전의 실제 지원 기능을 확인하고, 가능 여부를 확인하지 못하면 그 상태를 명시해줘.

직접 캡처 연계가 불가능하면 Packet Tracer에서는 Simulation/PDU 증거를, 별도의 허가된 실습 환경에서는 실제 .pcapng를 확보하는 방법을 안내해줘. 별도 환경에서 재현한 패킷을 원래 Packet Tracer 내부 트래픽이라고 표현하지 마. 두 환경의 구조 차이와 재현 가능한 범위를 기록해줘.

파일과 도구에 접근할 수 있으면 계획에서 멈추지 말고 구현해줘. 접근할 수 없으면 파일별 완성 코드와 실행 절차를 제공해줘. 사람이 해야 할 캡처나 팀의 입력이 필요한 단계만 대기 상태로 남기고, 수행하지 않은 실습을 완료했다고 주장하지 마.

04. 실제 Baseline·장애·복구 분석

정상 Baseline

다음 테스트마다 실행 위치, 캡처 위치, 발생시킬 트래픽, OS별 실행 방법, Wireshark display filter, 기대하는 정상 흐름, 저장할 증거를 안내해줘.

- 같은 VLAN·같은 Subnet의 PC 간 통신.
- PC와 Gateway 간 통신.
- 서로 다른 VLAN 간 통신.
- PC와 서버 IP 간 통신.
- DNS 이름 조회.
- 서버 TCP/Web 연결.

캡처를 시작한 뒤 테스트를 실행하고, 충분한 관찰 시간과 테스트 시각을 기록하게 해줘. display filter와 capture filter를 구분해줘. 기본 display filter인 arp, icmp, dns, tcp, tcp.flags.syn == 1, http의 용도와 한계를 설명해줘. tcp.flags.syn == 1은 SYN-ACK도 포함하므로 초기 SYN 집계와 구분해줘.

ARP/DNS 캐시, 기존 TCP 연결, 브라우저 캐시로 일부 단계가 안 보일 수 있음을 설명해줘. 캐시 조작이 필요하다면 OS·버전을 확인한 뒤 안내해줘. 정상·장애·복구 상태에서 테스트 대상, 캡처 지점, 필터, 관찰 시간, 캐시 조건을 가능한 한 동일하게 유지해줘.

원인을 모르는 장애 분석

SRE가 준 익명 case_id와 증상으로 시작해줘. 실제 원인이나 변경한 설정을 먼저 요청하지 마. 실습 대상으로 선정된 Gateway 오류, Access VLAN 오류, Trunk VLAN 누락, SVI Down, DNS 오류, Subnet Mask 오류, 서버 TCP Port 장애를 분석해줘.

항상 “실행한 테스트 → 관찰한 패킷 → 정상과 차이 → 가능한 원인 후보 → 필요한 추가 증거” 순서로 기록해줘. ARP Reply 부재로 VLAN 오류를, SYN 재전송으로 서비스 중단을 곧바로 확정하지 마. DNS 무응답·오류 응답·잘못된 응답, TCP 무응답·RST 응답을 구분해줘.

단일 캡처 지점의 한계를 기록해줘. 다른 지점의 관찰 없이 반대편에 패킷이 도달하지 않았다고 단정하지 마. Access Port 캡처에서 VLAN 태그가 없다는 사실을 VLAN 장애의 증거로 삼지 마. “관찰 구간에 응답이 보이지 않음”과 “응답 자체가 존재하지 않음”을 구분해줘.

복구 후 검증

SRE가 복구한 뒤 동일 테스트를 반복하고 Baseline / Fault / Recovered 상태를 비교해줘. 복구 설정의 작성과 실행은 SRE 범위로 두고, 나는 패킷 변화와 검증 결과를 기록한다. 최소 한 건은 내 분석 근거와 팀의 복구 검증에 어떻게 기여했는지 설명할 수 있게 정리해줘.

packet_analysis.md의 사례별 템플릿에는 사례 ID, 증상, 환경, 테스트, 캡처 출처와 위치, 프레임 번호·시각, 정상 대비 차이, 원인 후보, 배제하지 못한 가능성, 추가 확인, 복구 후 증거, 다음 담당자에게 전달한 내용을 포함해줘.

05. 파일 구조와 데이터 계약

기존 저장소에 맞게 조정하되 아래 구조를 제안해줘. 과제가 예시로 든 파일과 이번 요청으로 추가하는 보조 자료를 구분해줘.

- 02_packet_capture/README.md: 내 역할, 실행 순서, 협업 전달 방법.
- 02_packet_capture/capture_checklist.md: 캡처 전후 체크리스트.
- 02_packet_capture/wireshark_filters.md: 필터, 목적, 확인할 필드와 한계.
- 02_packet_capture/normal.pcapng: 실제 정상 캡처 후 생성.
- 02_packet_capture/fault_<case_id>.pcapng: 실제 장애 캡처 후 생성.
- 02_packet_capture/recovered_<case_id>.pcapng: 복구 후 실제 캡처를 확보한 경우.
- 02_packet_capture/packet_analysis.md: 증거 기반 분석 기록.
- 02_packet_capture/packet_summary.json: 실제 분석 데이터, 3번에게 전달.
- 02_packet_capture/packet_summary.example.json: 교육용 예제 데이터.
- 02_packet_capture/summarize_pcap.py: 필요한 경우에만 만드는 추출 보조 도구.
- 02_packet_capture/learning_lab/: 팀원용 인터랙티브 학습 웹앱.

실제 캡처가 없으면 가짜 .pcapng나 빈 캡처 파일을 만들지 마. 실제 packet_summary.json도 완료된 분석처럼 채우지 마. 준비 단계에서는 예제 파일과 템플릿을 제공하고 실제 증거 대기 상태를 표시해줘.

JSON 설계

과제의 단일 사례 객체 예시와 호환되도록 다음 필드를 유지해줘: case_id, source_ip, destination_ip, arp_request_count, arp_reply_count, icmp_request_count, icmp_reply_count, dns_query_count, tcp_syn_count, notes.

추가 필드로 schema_version, capture_file, evidence_source, capture_point, test_description, analysis_scope, dns_response_count, tcp_syn_ack_count, tcp_rst_count, evidence, limitations를 제안해줘. 여러 사례를 담는 방법은 AI Engineer와 합의하고, 기존 단일 객체 형식을 설명 없이 배열로 바꾸지 마.

- evidence_source는 교육용·실제 캡처·Packet Tracer 관찰을 구분한다.
- analysis_scope에는 시간 구간, 필터, 대상 흐름, 집계 기준을 기록한다.
- evidence에는 실제로 제공된 프레임 번호·시각·관찰 내용을 연결한다. 프레임이 없는 Simulation 증거에는 해당 출처의 이벤트 식별자를 사용한다.
- count는 해당 범위의 패킷 수인지 고유 트랜잭션 수인지 명시한다. 재전송 포함 여부도 정의한다.
- icmp_request_count와 icmp_reply_count는 Echo Request/Reply인지 명시하고 다른 ICMP 유형과 혼합하지 않는다.
- tcp_syn_count는 SYN=1, ACK=0인 패킷으로 정의하고 SYN-ACK와 분리한다.
- ARP 집계는 최종 목적지 IP뿐 아니라 실제 next hop과 연관된 ARP도 고려한다.
- 0은 관찰 가능한 범위를 분석했지만 해당 패킷이 없을 때만 사용한다. 미수집·미분석·관찰 불가능은 null과 이유로 구분한다.
- notes와 관찰 evidence에는 관찰 사실을 적는다. 가설은 분석 문서의 별도 영역에 둔다.
- 실제 장애 이름, SRE의 정답, 원인을 암시하는 파일명을 Blind Fault의 AI 입력에 넣지 않는다. 정답 대조는 분석 완료 후 별도 기록으로 남긴다.

필드 설명, 유효한 예제 JSON, 검증 규칙, AI Engineer가 읽는 방법을 함께 제공해줘. 실제 값이 아닌 예제에는 교육용 표시를 넣어줘.

선택적 추출 코드

실제 .pcapng를 읽고 특징을 추출하는 최소한의 보조 도구만 만들어줘. tshark 등 적합한 방법과 선택 이유를 설명하고 입력 경로, case_id, 분석 범위를 받게 해줘. 도구 미설치, 잘못된 파일, 빈 캡처를 명확히 처리해줘. Wireshark의 동일 필터로 수동 집계한 결과와 비교하는 검증 절차를 제공해줘. 합성 테스트 데이터는 실제 실습 증거와 분리하고, 실행하지 않은 검증을 성공했다고 표현하지 마.

06. 팀원 중심의 학습 UX

대상 사용자는 이 프로젝트를 처음 보는 팀원이다. ARP, VLAN, DNS, TCP를 들어봤어도 실제 연결 관계는 모를 수 있다. 사용자가 다음 질문에 답할 수 있게 설계해줘.

- 지금 어떤 통신을 시도하고 있는가?
- 패킷은 어디에서 어디로 이동하며 이 단계가 왜 필요한가?
- 정상이라면 다음에 무엇이 보여야 하는가?
- 지금 관찰한 사실은 무엇이며 아직 모르는 것은 무엇인가?
- 어떤 원인 후보가 있고 무엇을 추가 확인하면 구분할 수 있는가?

설명은 “쉬운 설명 → 실제 흐름 → 기술적 세부사항” 순서로 제공해줘. 비유를 쓰면 실제 동작과 비유의 차이를 밝혀줘. 처음 등장하는 용어를 화면 안에서 설명하고 중요한 근거를 톨팁에만 숨기지 마. 모든 단원은 “예측 → 조작 → 관찰 → 설명”의 학습 흐름을 따른다.

첫 화면에는 목적, 배우게 될 내용, 정상 패킷 따라가기, 장애 조사하기를 제공해줘. 긴 소개문보다 시작 행동을 분명히 해줘. 언제든지 현재 목표·진행 단계·다음 행동이 보여야 한다.

기본 사용자 흐름은 시작하기 → 정상 패킷 따라가기 → 정상·장애 비교 → 설정 변경 실험 → Blind Fault 조사 → 실제 증거 읽기 → JSON 전달 → 이해 확인이다. 발표용으로 짧은 경로를, 자세한 학습용으로 설명을 펼쳐 볼 수 있는 경로를 제공해줘.

기본 화면 구성

- 상단: 현재 단원, 학습 목표, 예제/실제 데이터 표시.
- 중앙 왼쪽: 클릭 가능한 PC, Switch, Gateway, Server 토폴로지와 패킷 경로.
- 중앙 오른쪽: 지금 일어나는 일, 그 이유, 다음에 기대하는 패킷.
- 제어 영역: 이전 단계, 다음 패킷, 재생, 일시정지, 속도, 초기화.
- 하단: 시간·출발지·목적지·프로토콜·요약이 있는 패킷 목록.
- 상세 영역: 선택한 패킷의 쉬운 설명, 관련 필드, 해석의 한계.

토폴로지, 패킷 목록, 설명 패널을 연동해줘. 패킷 행을 선택하면 이동 경로와 관련 필드가 강조되어야 한다. 자동 재생 외에 한 단계씩 읽으며 진행하는 방식을 기본 제공해줘. 색상만으로 상태를 구분하지 말고 이름·아이콘·텍스트를 함께 사용해줘.

07. 정상 통신을 이해하는 인터랙티브 예시

A. 같은 Subnet의 PC 간 통신

목적지가 같은 Subnet인지 판단하는 과정, 필요한 경우 ARP Request/Reply, ICMP Echo Request/Reply를 보여줘. 이 예제의 조건에서 Gateway를 거치지 않는 이유와 ARP Broadcast 범위를 설명해줘. “다음으로 알아야 할 주소는 무엇일까?” 같은 예측 질문을 넣어줘.

B. 다른 Subnet의 서버와 통신

최종 목적지와 next hop을 구분하고, 원격 서버가 아니라 Gateway의 MAC을 알아내는 과정을 보여줘. IP 주소와 MAC 주소를 나란히 표시해 라우터 전후의 L2 정보 변화를 이해하게 해줘. 이 예제는 NAT를 사용하지 않는다고 명시하고, 목적지 IP와 링크별 MAC의 역할을 구분해줘.

C. 도메인으로 웹 서버 접속

DNS Query/Response → 서버 IP 확인 → TCP SYN/SYN-ACK/ACK → HTTP Request/Response를 단계별로 보여줘. DNS, TCP, HTTP가 각각 해결하는 문제를 설명해줘. 교육용 예제는 일반 DNS와 HTTP를 사용한다고 밝히고 실제 HTTPS나 암호화된 DNS에서는 같은 형태의 내용이 보이지 않을 수 있음을 짧게 안내해줘.

각 예시는 시작 전 예측 질문, 단계별 설명, 선택 가능한 패킷 상세, 짧은 이해 확인을 포함해줘. ARP/DNS 캐시 유무를 조절하면 일부 단계가 생략되는 것을 보여줘. 관찰되지 않은 패킷이 언제나 장애를 의미하지는 않는다는 점을 학습하게 해줘.

08. 비교 화면과 설정 변경 실험실

정상·장애 비교

동일 테스트의 정상과 장애를 나란히 보여주고 작은 화면에서는 탭 또는 위아래 비교를 제공해줘. 동일 단계 기준으로 진행하고 첫 차이, 정상에서 기대한 응답, 장애에서 관찰한 현상을 강조해줘. 캡처 위치와 관찰 시간, 관찰 사실과 원인 후보를 구분해 표시해줘.

교육 사례는 다음을 포함하되 실제 과제 사례와 혼동하지 마.

- 잘못된 Gateway를 향한 ARP와 응답 부재.
- Access VLAN 오류와 Trunk VLAN 누락.
- SVI Down과 Gateway 응답 부재 가능성.
- DNS 무응답과 오류 응답의 차이.
- TCP SYN 무응답과 RST 응답의 차이.
- Subnet Mask 오류로 달라지는 next-hop 판단.

각 예시가 성립하는 주소·연결·서비스·필터 조건을 먼저 정의해줘. 특정 설정 오류가 항상 같은 패킷을 만든다고 표현하지 마. VLAN 오류, Trunk 누락, SVI Down처럼 한 지점에서 비슷하게 보이는 사례는 추가 증거가 왜 필요한지 설명해줘.

설정 변경 실험실

PC Gateway, Subnet Mask, Access VLAN, Trunk 허용 VLAN, SVI 상태, DNS 응답 상태, Web Port 상태를 교육용 환경에서 조절하게 해줘. 기본은 한 번에 하나의 변수를 바꾸는 방식이다.

상호작용은 설정 변경 → 결과 예측 → 통신 실행 → 패킷 변화 관찰 → 이유 설명 → 정상 복원 순서로 구현해줘. 설정에 따라 패킷 순서, 응답 여부, 다음 단계 진행 가능성과 설명이 일관되게 달라져야 한다. 버튼을 눌러도 정해진 문장만 바뀌는 장식적인 데모로 만들지 마.

완전한 네트워크 에뮬레이터까지 만들 필요는 없다. 지원 시나리오와 단순화한 가정을 명시하고 그 범위에서는 정확하게 동작하는 교육용 모델을 구현해줘. 교육용 설정 조작이 실제 장비 설정을 바꾸지 않도록 분리해줘.

09. Blind Fault 조사 게임

학습 모드에서는 전체 원인과 흐름을 설명하고, Blind Fault 모드에서는 원인을 숨긴 증상과 관찰 가능한 증거만 보여줘. 실제 장애 원인은 SRE가 관리하며 웹앱에 미리 제출하도록 요구하지 마. 교육용 Blind Fault는 별도의 예제임을 표시해줘.

진행 순서는 증상 확인 → 검사 선택 → 결과와 패킷 관찰 → 증거 노트 추가 → 후보 선택 → 후보를 구분할 추가 검사 → 결론 제출 → 해설과 복구 전후 비교다.

선택 가능한 검사는 같은 VLAN PC ping, Gateway ping, 서버 IP ping, DNS 조회, TCP 연결 시도, 다른 지점의 캡처 확인, 필요한 장비 상태 확인으로 구성해줘. 패킷을 먼저 관찰하고, 장비 상태 확인은 남은 가설을 구분하는 단계로 자연스럽게 유도해줘. 선택한 검사에 해당하는 결과만 공개해줘.

사건 제목, 파일명, 처음부터 빨간색인 장애 장치, 톨팁, 접근성 레이블 등으로 정답을 미리 노출하지 마. 정적 웹앱의 클라이언트 코드에 들어 있는 교육용 정답은 개발자 도구로 볼 수 있으므로 보안 시험 도구라고 주장하지 마. 실제 SRE 정답은 AI 전달 데이터나 학습 앱에 섞지 마.

평가는 속도나 정답 이름보다 관련 증거 수집, 관찰과 추측의 구분, 대안 검토, 유효한 추가 검사를 중심으로 해 줘. 시간 제한은 기본으로 두지 마. 오답에는 근거가 부족한 이유와 다음 관찰 방향을 알려줘. 힌트는 단계적으로 제공하고 재시도와 초기화를 지원해줘.

원인이 여러 개 가능한 증거에서 단일 정답을 강요하지 마. 추가 증거가 나올 때까지 합리적인 복수 후보를 인정 해줘. 복구 후에는 같은 검사를 반복해 무엇이 달라졌는지 직접 비교하게 해줘.

10. 실제 증거 읽기와 팀원 전달

프로토콜마다 Wireshark display filter, 확인할 필드, 필드의 의미, 흔한 오해, 증거 기록 예시를 보여줘. 교육용 패킷 표와 실제 Wireshark 화면의 대응 관계를 설명하되 실제 캡처 화면이 없으면 재현 UI라고 표시해줘.

packet_summary.json 불러오기 기능을 제공하고 지원 스키마를 검증해줘. 누락된 필드, 잘못된 타입, 지원하지 않는 버전은 이해하기 쉬운 오류로 안내해줘. JSON에 없는 프레임·시각·토폴로지·패킷 세부내용을 생성해서 실제 사실처럼 채우지 마. 요약 데이터만 있으면 요약 수준으로 표시하고 없는 정보는 제공되지 않음으로 표시해 줘.

브라우저의 PCAP 파싱을 구현하지 않았다면 .pcapng를 직접 분석할 수 있다고 표시하지 마. 실제 캡처 → 추출 보조 도구 → JSON → 학습 웹앱이라는 연결을 안내해줘. Packet Tracer 이벤트와 실제 캡처는 각기 출처를 표시 해줘.

network_spec.md → 캡처와 분석 → packet_summary.json → AI Engineer의 diagnosis.json → SRE의 복구 검증 흐름을 시각화해줘. 패킷 증거를 선택하면 관련 JSON 필드가 강조되게 해줘. ARP Reply 0건, 미수집으로 null, 관찰 지점 한계로 판단 불가능한 경우를 비교하는 예시를 제공해줘.

내보내기는 관찰 사실, 분석 범위, 출처, 한계를 보존해줘. 교육용 내보내기에는 예제 표시를 유지하고, 실제 정답이나 추측을 관찰 count에 섞지 마. 원본 데이터와 사람이 추가한 메모의 구분도 유지해줘.

11. 설명 품질, 접근성과 이해 확인

한국어를 읽기 편한 글자 크기·행간, 일관된 장치 이름과 프로토콜 표기를 사용해줘. 키보드 조작, 명확한 포커스, 모션 감소 설정, 색상 외의 상태 표현을 제공해줘. 노트북과 모바일에서 내용이 잘리지 않게 하고, 패킷 표는 필요한 경우 명확한 가로 스크롤을 제공해줘.

긴 설명은 단계적으로 펼치되 현재 판단에 필요한 핵심 근거는 항상 보이게 해줘. 재생·정지·단계 이동·초기화가 언제나 예측 가능하게 동작해야 한다. 장식보다 사용자의 이해에 도움이 되는 움직임을 우선해줘. 외부 JSON의 텍스트를 실행 가능한 HTML로 삽입하지 마.

각 단원의 끝에는 팀원이 자기 말로 설명할 질문을 제공해줘.

- 다른 Subnet의 서버에 접속할 때 왜 Gateway의 MAC이 필요한가?
- ping이 성공해도 웹 접속은 왜 실패할 수 있는가?
- DNS 응답이 없다는 사실만으로 DNS 서버 장애를 확정할 수 있는가?
- ARP Reply가 보이지 않았다는 사실만으로 알 수 없는 것은 무엇인가?
- 지금 남은 두 후보를 구분하려면 무엇을 검사해야 하는가?
- 내 캡처가 보여주는 범위와 보여주지 못하는 범위는 무엇인가?

자유 서술 답변을 단순 키워드 일치로 정확하다고 판정하지 마. 모범 설명, 포함하면 좋은 근거, 자기 점검 기준을 제공해줘. 실제 학습 효과는 팀원의 사용과 설명을 통해 확인해야 하며, UI 구현만으로 이해가 검증되었다고 주장하지 마.

팀 발표 전체가 5~8분인 과제 흐름을 고려해 내 파트는 짧은 패킷 증거 시연 경로를 제공해줘. 자세한 학습 경로는 별도로 유지하고 내 파트만으로 전체 발표 시간을 소모하지 않게 해줘.

12. 구현 순서, 검증과 최종 전달

기존 저장소에 스택이 있으면 따르고, 없으면 HTML/CSS/JavaScript의 간단한 정적 웹앱을 우선해줘. 외부 API, 로그인, 유료 서비스 없이 교육 예제 모드가 동작해야 한다. 실제 .pcapng 분석은 환경과 도구가 준비된 경우에만 수행해줘.

첫 응답에서는 역할과 완료 기준, 핵심 질문, 1번·4번에게 받을 정보 체크리스트, 폴더 구조, 학습 흐름, 인터랙션별 “행동 → 화면 변화 → 배우는 내용”, 데이터 계약과 기술 가정을 짧게 제시해줘. 이후 준비 문서와 실행 가능한 구현을 진행해줘.

먼저 “정상 캡처 1건 → 장애 캡처 1건 → 비교 → JSON 전달”의 실제 작업 경로와 “정상 흐름 → 장애 조사 → 해설”의 교육 경로를 각각 완결되게 연결해줘. 실제 캡처를 기다리는 동안 예제 구현을 진행하고 나머지 교육 사례로 확장해줘. 일부 데모만 만들고 전체 요청을 완료했다고 하지 마.

다음 사항을 기능과 데이터 수준에서 검증해줘.

- 정상·장애 예시의 패킷 순서가 네트워크 조건과 설명에 일치하는가?
- 사용자 설정에 따라 결과가 실제로 달라지는가?
- 캐시 유무, SYN/SYN-ACK/RST, DNS 응답 유형이 일관되게 처리되는가?
- 초기화 후 이전 사례의 상태나 정답이 남지 않는가?
- Blind Fault에서 정답이 화면에 미리 노출되지 않는가?
- JSON의 0과 null, 실제와 예제가 구분되고 잘못된 입력을 처리하는가?
- JSON에 없는 패킷 세부사항을 만들어내지 않는가?
- 키보드·작은 화면·모션 감소 환경에서 핵심 작업을 수행할 수 있는가?
- 실제 집계는 Wireshark의 동일 범위 수동 확인과 일치하는가?
- 팀원이 사전 설명 없이 시작하고 “증거 → 해석 → 추가 확인”을 설명할 수 있는지 점검할 절차가 있는가?

최종 결과에는 실행 가능한 웹앱, 실행 방법, 분석 문서와 데이터 템플릿, 구현된 사례·인터랙션 목록, 실제 캡처 연결 방법, 팀원용 짧은 시연 순서, 검증 결과와 남은 한계를 포함해줘. 완료한 개발 작업과 아직 사람이 수행해야 할 캡처·팀 합의·사용성 확인을 명확히 구분해줘.

과제 준수 점검도 짧게 남겨줘. 실제 정상·장애 증거가 있는지, 패킷을 먼저 관찰했는지, AI에 정답을 먼저 알려주지 않았는지, 내 출력이 다음 담당자의 입력으로 연결되는지, 역할을 침범한 기능이 없는지. 교육용 시뮬레이션은 실제 과제 증거의 대체물이 아니라 이해를 돕는 설명 자료라는 원칙을 끝까지 지켜줘.